



Laboratorio 1

Codificador de Fuente sin Pérdidas

Docente: Sergio Cabarrouy
Integrantes: Gabriel Perez, Fabrizio De Santis y Gaston
Lopes

Introducción	3
Parte 1 — Análisis de información de una fuente	3
Descripción del código y resultados	3
Parte 2 — Modelado de Markov y tasa de entropía	4
Descripción del código y resultados	4
Descripción del código y resultados	5
Conclusiones y observaciones:	9

Introducción

El presente laboratorio tiene como propósito aplicar los conceptos fundamentales de la teoría de la información y la compresión sin pérdidas mediante el desarrollo e implementación de distintos métodos de codificación. Se busca analizar el comportamiento de fuentes de información y evaluar la eficiencia de diversos algoritmos de compresión.

El objetivo principal consiste en implementar, verificar y comparar experimentalmente técnicas estudiadas en el curso, incluyendo el cálculo de medidas de información (entropía, entropía condicional e información mutua), el modelado de fuentes con memoria mediante cadenas de Markov, y la implementación de algoritmos como Huffman y un segundo método adicional (codificación aritmética o LZW), además de contrastar los resultados con herramientas estándar de compresión.

Parte 1 — Análisis de información de una fuente

Descripción del código y resultados

En la primera sección se desarrolla un conjunto de funciones en Python orientadas al análisis estadístico de fuentes de información, trabajando a nivel de bytes y de caracteres.

En la **subparte 1.1**, el código implementa el cálculo de la distribución empírica de probabilidad de los símbolos de un archivo, considerando los 256 posibles valores de byte. A partir de esta distribución, se obtiene la entropía empírica de orden 0 y se genera un histograma con los 20 símbolos más frecuentes. Los resultados reflejan la no uniformidad de los símbolos en textos reales, lo que se traduce en valores de entropía inferiores al máximo teórico (8 bits por símbolo), confirmando que existe redundancia explotable para compresión.

En la **subparte 1.2**, se extiende el análisis al caso de pares de símbolos consecutivos, construyendo la distribución conjunta y calculando la entropía conjunta, la entropía condicional y la información mutua. Los resultados obtenidos muestran que la entropía condicional es menor que la entropía simple, lo que indica que el conocimiento del símbolo anterior reduce la incertidumbre del siguiente. Asimismo, los valores positivos de información mutua evidencian dependencia estadística entre símbolos consecutivos, en concordancia con la teoría de fuentes con memoria. Se observa además una mayor dependencia en el texto en español respecto al inglés, reflejada en mayor información mutua y menor entropía condicional.

En la **subparte 1.3**, el código genera dos fuentes sintéticas: una fuente sin memoria (DMS) y una fuente de Markov de orden 1, ambas con parámetros predefinidos. Posteriormente, se calculan sus entropías empíricas y, en el caso de la fuente de Markov, su entropía condicional, comparándolas con los valores teóricos. Los resultados muestran una buena aproximación entre los valores empíricos y teóricos, validando la correcta implementación y confirmando que las fuentes con memoria presentan una tasa de entropía menor que su entropía marginal, debido a la dependencia entre símbolos.

En conjunto, los resultados obtenidos verifican empíricamente los principios teóricos de la teoría de la información, mostrando cómo la estructura estadística de una fuente influye directamente en su incertidumbre y en su potencial de compresión.

Parte 2 — Modelado de Markov y tasa de entropía

Descripción del código y resultados

En esta sección se desarrolla un análisis basado en modelos de Markov de distintos órdenes para estudiar cómo la memoria de la fuente influye en la entropía y, por tanto, en el potencial de compresión.

En la **subparte 2.1**, el código implementa la estimación empírica de las transiciones de Markov para órdenes $k = 0, 1, 2, 3, 4$. A partir del texto, se construyen los conteos de contextos y de símbolos siguientes, lo que permite calcular la tasa de entropía empírica H_k . Este procedimiento sigue directamente la definición teórica, donde la entropía se obtiene ponderando la incertidumbre condicional de cada contexto por su probabilidad de ocurrencia.

En la **subparte 2.2**, se calculan y comparan las tasas de entropía para distintos valores de k en textos en inglés y español, representando los resultados gráficamente. Se verifica empíricamente que la secuencia $H_0 \geq H_1 \geq H_2 \geq \dots$ es no creciente, lo cual concuerda con la teoría: a medida que se incorpora más contexto, se reduce la incertidumbre sobre el siguiente símbolo. Además, se observa que la entropía decrece más rápidamente en el texto en inglés, lo que indica que su estructura presenta patrones más predecibles con contextos relativamente cortos, mientras que el español mantiene mayor variabilidad en órdenes más altos.

En la **subparte 2.3**, se utiliza la tasa de entropía de orden 4 para estimar el límite teórico de compresión de los textos. A partir de estos valores, se calcula el tamaño mínimo alcanzable sin pérdida y el ratio de compresión teórico. Los resultados muestran ratios aproximados de 31:1 para el inglés y 23:1 para el español, lo que refleja una mayor predictibilidad del texto en inglés en modelos de orden alto.

En conjunto, los resultados confirman los principios teóricos de las fuentes con memoria: la entropía disminuye al considerar dependencias entre símbolos, y la tasa de entropía condicionada define el límite fundamental de compresión. Asimismo, evidencian cómo las diferencias estructurales entre idiomas impactan directamente en su capacidad de ser comprimidos.

Parte 3 — Codificador/decodificador de Huffman

Descripción del código y resultados

En esta sección se implementa el algoritmo de Huffman completo, incluyendo codificación, decodificación y verificación de correctitud, trabajando a nivel de bytes sobre los cuatro archivos de prueba.

En la subparte 3.1, el código construye el árbol de Huffman a partir de las frecuencias empíricas de cada byte, utilizando una cola de prioridad implementada con el módulo `heapq`. El algoritmo extrae iterativamente los dos nodos de menor frecuencia, los une bajo un nodo padre cuya frecuencia es la suma de ambos, y reinsertar el resultado hasta obtener un único árbol. A partir de este árbol se genera la tabla de códigos mediante un recorrido en profundidad, asignando el bit 0 a la rama izquierda y el bit 1 a la rama derecha. El stream comprimido resultante incluye un encabezado con la tabla de frecuencias serializada, lo que permite que el decodificador sea completamente autónomo.

En la subparte 3.2, se verificó que la decodificación del stream comprimido reproduce exactamente los datos originales en los cuatro archivos de prueba, sin pérdida de ningún byte. Adicionalmente, se calculó la suma de Kraft obteniendo un valor de 1.000000 en todos los casos, lo que confirma que los códigos generados forman un código prefijo completo y válido.

En la subparte 3.3, se calcularon las métricas de compresión para cada archivo. Los textos en español e inglés alcanzaron ratios de compresión de 1.79 y 1.81 respectivamente, mientras que las fuentes sintéticas lograron mayores ratios: 2.06 para la fuente DMS y 2.42 para la fuente de Markov. Esto se explica por la mayor concentración de la distribución de bytes en las fuentes sintéticas, lo que permite asignar códigos más cortos a los símbolos dominantes. La eficiencia del código superó el 98% en todos los archivos, con una redundancia inferior a 0.06 bits por símbolo respecto al límite teórico. Se verificó además que la desigualdad $H(X) \leq L_H < H(X)+1$ se cumple en todos los casos, confirmando la garantía teórica del algoritmo de Huffman.

En conjunto, los resultados muestran que Huffman se aproxima al límite teórico de compresión de orden 0 con alta eficiencia. Sin embargo, al operar sobre símbolos individuales sin considerar dependencias entre ellos, no aprovecha la memoria de las fuentes. Esto es especialmente evidente en la fuente de Markov, donde un codificador que explote la estructura estadística entre símbolos consecutivos, como LZW, podría obtener mejores resultados, tal como se analiza en la Parte 4.

Parte 4 — Codificador adicional

Se implementó un codificador y decodificador sin pérdidas basado en el algoritmo LZW (Lempel–Ziv–Welch). El objetivo principal fue analizar el comportamiento de un método de compresión basado en diccionarios dinámicos y compararlo posteriormente con el algoritmo de Huffman de la Parte 3.

Inicialmente se construyó un diccionario con las 256 posibles entradas correspondientes a los valores de byte individuales. Durante el proceso de codificación, el algoritmo detecta secuencias repetidas y las incorpora dinámicamente al diccionario, asignándoles nuevos códigos.

Para la decodificación se implementó el proceso inverso, reconstruyendo el archivo original a partir de los códigos comprimidos. También se manejó el caso especial característico de LZW, correspondiente a la situación en la que el decodificador recibe un código aún no agregado al diccionario..

Además, se realizaron pruebas utilizando dos tamaños máximos de diccionario diferentes (4096 y 65536 entradas), con el objetivo de analizar el impacto de esta configuración en el ratio de compresión. Los resultados obtenidos mostraron que ambos tamaños produjeron prácticamente el mismo desempeño para el archivo analizado, indicando que el tamaño menor del diccionario fue suficiente para capturar los patrones relevantes presentes en el texto.

Parte 5 — Comparativa y análisis

Método	Tamaño	Ratio	Codif(s)	Decodif(s)
Huffman	6619	0.5399	0.003607	0.007006
LZW	6844	0.5582	0.007568	0.003228
gzip	778	0.0635	0.000756	0.000288
bz2	1282	0.1046	0.003577	0.000374
lzma	832	0.0679	0.004751	0.000216

Captura de los resultados finales

En la tabla comparativa se puede observar el tamaño comprimido, el ratio de compresión y los tiempos de codificación y decodificación de cada uno de los compresores.

De la tabla comparativa, gzip obtuvo el ratio de compresión más bajo, por lo que comprimió mejor, esto se debe a que utiliza Deflate = LZ77 + Huffman (Detecta patrones repetitivos y codifica eficientemente)

Comparando las implementaciones, se observa que Huffman obtuvo mejor ratio que LZW, obteniendo un ratio de 0,5399 frente a 0,5582.

Se observó que lzma requirió un mayor tiempo de codificación.

Conclusiones y observaciones:

- 1) La implementación de los algoritmos de Huffman y LZW permitió verificar los principios de la compresión sin pérdidas vistos en el curso, en ambos casos se logró la reconstrucción de los datos originales.
- 2) Se compararon las implementaciones con herramientas como gzip, bz2 y lzma, observando que éstos logran ratios de compresión mejores